

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Minimax Search:** A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning:** A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Turing Test:** An operational test for intelligent behavior, the Imitation Game, where a human interrogator tries to distinguish between a human and a machine based on their text-based responses.
- **Game Tree:** A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing outcomes.
- **Perceptron:** A fundamental unit in neural networks that takes multiple inputs, applies weights to them, sums the weighted inputs, and produces an output based on a threshold.
- **Reinforcement Learning:** A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.
- **Matrix:** A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform:** A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c .
- **Matrix Multiplication:** An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of the i -th row of the first matrix and the j -th column of the second matrix. The inner dimensions must match.
- **Matrix Inverse:** For a square matrix A , its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition:** An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T .
- **Derivative:** A measure of how a function changes with respect to a change in its input.
- **Partial Derivative:** A derivative of a function of multiple variables with respect to one of those variables, holding the others constant.
- **Sigmoid Function:** A mathematical function that maps any input value to an output value between 0 and 1. Often used as an activation function in neural networks.
- **Inductive Reasoning:** The process of deriving general principles from specific observations.
- **Linear Regression:** A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.

- **Model:** A mathematical representation of a system or process, used to make predictions or understand relationships between variables.
- **Overfitting:** A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Loss Function:** A function that quantifies the difference between the predicted and actual values in a machine learning model.
- **Decision Boundary:** A line or surface that separates different classes of data points in a classification problem.
- **Stochastic Gradient Descent:** An iterative optimization algorithm that updates model parameters using the gradient calculated from a randomly selected subset of the training data.
- **Logistic Regression:** A statistical model that uses a logistic function to model the probability of a binary outcome.
- **Empirical Risk Minimizer:** A learning principle that aims to find the model that minimizes the average loss over the training data.
- **Gradient Descent:** An iterative optimization algorithm that updates model parameters by moving in the direction of the negative gradient of the loss function.
- **Perceptron Loss:** A loss function used in the perceptron algorithm, which is zero if the prediction is correct and equal to the negative of the margin otherwise.
- **Hinge Loss:** A loss function used in support vector machines (SVMs), which is zero if the margin is greater than 1 and equal to 1 - margin otherwise.
- **Logistic Loss:** A loss function used in logistic regression, which is based on the log-likelihood of the data given the model parameters.
- **Multiple Linear Regression:** A statistical method used to model the relationship between a dependent variable and two or more independent variables.
- **Binary Classification:** A machine learning task where the goal is to classify data points into one of two categories.
- **Image Classification:** The task of assigning a predefined category label to an image.
- **Semantic Gap:** The difference between how humans perceive images (semantically) and how computers represent them (numerically).
- **Viewpoint Variation:** The challenge in image classification caused by changes in the camera's or object's position and orientation.
- **Illumination:** The challenge in image classification caused by variations in lighting conditions.
- **Deformation:** The challenge in image classification caused by changes in the object's shape or pose.
- **Occlusion:** The challenge in image classification where parts of an object are hidden or obscured.
- **Background Clutter:** The challenge in image classification where distracting elements in the background hinder object recognition.
- **Invariant Features:** Features that remain consistent across different variations of an object (e.g., different viewpoints, lighting, or poses).
- **Bag of Features:** An image representation method where an image is represented by the frequencies of "visual words" (local features).

- **Visual Words:** Basic visual elements extracted from images, representing key features used in Bag of Features.
- **Textons:** Basic elements or features that characterize texture.
- **Nearest Neighbor (NN) Classifier:** A classification algorithm that assigns the label of the nearest training data point to each test data point.
- **Noise:** Random variations or unwanted disturbances in image data.
- **Salt and pepper noise:** Random occurrences of black and white pixels in an image.
- **Impulse noise:** Random occurrences of white pixels in an image.
- **Gaussian noise:** Variations in intensity drawn from a Gaussian normal distribution.
- **Mean Filtering:** A noise reduction technique that replaces each pixel with the average of its neighboring pixels.
- **Moving Average:** A method for smoothing data by averaging values within a sliding window.
- **Weighted Moving Average:** A moving average where neighboring values are weighted differently.
- **Image Filtering:** The process of computing a function of the local neighborhood at each pixel in an image.
- **Identity Filter:** An image filter that leaves the image unchanged.
- **Sharpening:** An image enhancement technique that increases the contrast between edges and details.
- **Category Models:** Representations of different object categories in a model space, often based on feature vectors.
- **Codewords:** Visual features extracted from an image, often represented as small image patches.
- **Edge Detection:** A technique in image processing used to identify points of rapid change in image intensity.
- **Edges:** Curves in an image where brightness changes significantly.
- **Feature Vector:** A vector representing the features extracted from an image or data point.
- **Finite Difference:** A method for approximating the derivative of a function using discrete values.
- **Gaussian Pyramid:** A multi-resolution representation of an image, created by repeatedly blurring and subsampling the image.
- **Image Gradient:** A vector representing the direction and magnitude of the greatest rate of change in image intensity.
- **Intensity Function:** A function that maps each pixel in an image to its intensity value.
- **Intensity Profile:** A plot of intensity values along a scanline of an image.
- **Model Space:** A multi-dimensional space where each point represents a category model.
- **Perceptual Organization:** The process of grouping pixels into objects based on visual cues like edges.
- **Vision Pyramid:** A hierarchical representation of an image at multiple resolutions.
- **Convolutional Neural Network (CNN):** A type of artificial neural network designed to process data with a grid-like topology, such as images. CNNs use convolutional layers to extract features from the input data.

- **ImageNet**: A large dataset of images used for training and evaluating image recognition models. It contains millions of images across thousands of categories.
- **AlexNet**: One of the first deep convolutional neural networks to achieve high accuracy in image recognition competitions, notably the ImageNet challenge. It was a significant milestone in the development of deep learning.
- **ConvNet**: Short for Convolutional Neural Network.
- **Artificial Neuron**: A mathematical model of a biological neuron, used as a building block in artificial neural networks. It takes weighted inputs, sums them, and applies an activation function to produce an output.
- **Convolution Layer**: A layer in a convolutional neural network that applies a filter (kernel) to the input data to extract features. The filter slides across the input, performing element-wise multiplication and summation at each position.
- **Pooling Layer**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps, typically by applying a max or average pooling operation. This helps to reduce computational cost and make the network more robust to small variations in the input.
- **Convolution Filter**: A small matrix of weights used in a convolutional layer to extract features from the input data. The filter is convolved with the input data to produce an output feature map.
- **Multidimensional Convolution**: A convolution operation applied to input data with multiple channels (e.g., RGB channels in an image). Each channel is processed independently by a corresponding filter, and the results are combined.
- **Local Receptive Field**: The small region of the input data that a neuron in a convolutional layer is connected to. This leads to sparse connectivity and reduces the number of parameters in the network.
- **Sparse Connectivity**: A characteristic of convolutional neural networks where each neuron is connected to only a small subset of neurons in the previous layer, unlike fully connected networks.
- **Parameter Sharing**: A key characteristic of convolutional layers where the same set of weights (the filter) is used across all spatial locations in the input. This significantly reduces the number of parameters in the network.
- **Rectified Linear Unit (ReLU)**: A non-linear activation function defined as $f(x) = \max(0, x)$. It is commonly used in neural networks due to its computational efficiency and ability to mitigate the vanishing gradient problem.
- **Max Pooling**: A pooling operation that selects the maximum value within a defined region (e.g., 2x2) of the input feature map.
- **Average Pooling**: A pooling operation that calculates the average value within a defined region (e.g., 2x2) of the input feature map.
- **Back-propagation**: An algorithm used to train neural networks by calculating the gradient of the loss function with respect to the network's weights and then updating the weights to minimize the loss.
- **Fully Connected Layer (FC)**: A layer in a neural network where each neuron is connected to every neuron in the previous layer. This is typically the final layer before the output layer.
- **MNIST**: A large database of handwritten digits used for training and testing in machine learning, particularly in image recognition.
- **CIFAR-10**: A large dataset of 32x32 color images, commonly used for benchmarking image classification models. It contains 10 classes with 6000 images per class.

- **LeNet-5:** A convolutional neural network architecture specifically designed for the MNIST dataset. It consists of convolutional, pooling, and fully connected layers.
- **Training Data:** The subset of data used to train a machine learning model.
- **Held Out Data:** A subset of data used for hyperparameter tuning and model selection.
- **Test Data:** A subset of data used to evaluate the performance of a trained machine learning model on unseen data.
- **Evaluation Function:** A function used to assess the performance of a machine learning model.
- **Model Complexity:** A measure of the capacity of a model to fit complex patterns in data. High complexity can lead to overfitting.
- **Bias:** The error introduced by approximating a real-world problem with a simplified model. High bias leads to underfitting.
- **Variance:** The error introduced by the model's sensitivity to small fluctuations in the training data. High variance leads to overfitting.
- **Hyperparameters:** Parameters that control the learning process of a machine learning model, but are not learned from the data itself.
- **Regularization:** A technique used to prevent overfitting by adding a penalty term to the loss function that discourages large weights.
- **Regularization Coefficient (λ):** A hyperparameter that controls the strength of the regularization penalty.
- **Learning Curves:** Graphs that show the training and testing performance of a model as a function of the training set size or model complexity.
- **S-fold Cross Validation:** A model evaluation technique that divides the data into S folds, using each fold once as a validation set and the rest as a training set.
- **Data Analysis Pipeline:** The sequence of steps involved in collecting, preparing, modeling, and evaluating data.
- **Matrix Factorization:** A technique to decompose a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Principal Component Analysis (PCA):** A linear dimensionality reduction technique that identifies the principal components (directions of maximum variance) in a dataset to reduce dimensionality while preserving most of the data's variance.
- **Dimensionality Reduction:** Techniques that transform data from a higher-dimensional space to a lower-dimensional space, reducing the number of attributes while retaining important information.
- **Low Rank Matrix Approximations:** Approximating a high-rank matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Embeddings:** A low-dimensional representation of data points, often obtained through dimensionality reduction techniques like matrix factorization, that captures the essential relationships between data points.
- **Collaborative Filtering:** A recommender system technique that predicts user preferences based on the preferences of similar users.
- **Recommender Systems:** Systems that predict user preferences for items, such as movies or products, based on past user behavior and other relevant data.

- **EigenSNPs**: Eigenvectors obtained from Principal Component Analysis (PCA) applied to Single Nucleotide Polymorphisms (SNPs) data, representing the principal components of genetic variation.
- **Single Nucleotide Polymorphisms (SNPs)**: The most common type of genetic variation, representing locations in the genome where different nucleotide bases (A, C, G, T) are observed across individuals.
- **Bias-Variance Tradeoff**: The trade-off between model complexity and prediction accuracy. High complexity models can overfit the training data (high variance), while low complexity models may underfit (high bias).
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of a model's predictions, calculating the average squared difference between predicted and actual values.
- **Netflix Prize**: A competition offering a 1millionprizeforimprovingtheaccuracyofNetflix'smovie recommendationsystem.
- **Clustering**: The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **Supervised Learning**: A machine learning paradigm where the algorithm learns from labeled data (input-output pairs) to predict outputs for new inputs.
- **Unsupervised Learning**: A machine learning paradigm where the algorithm learns from unlabeled data to discover underlying patterns or structures.
- **K-means clustering**: A popular clustering algorithm that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster center or centroid).
- **Hamming distance**: A distance metric used for comparing strings of equal length. It counts the number of positions at which the corresponding symbols are different.
- **Matching coefficient**: A similarity measure between two binary vectors. It is calculated as the number of matching bits divided by the total number of bits.
- **Euclidean distance**: The straight-line distance between two points in Euclidean space. For two points x and y in p -dimensional space, it is defined as $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$.
- **Inertia**: In K-means clustering, the sum of squared distances of samples to their closest cluster center.
- **Purity**: A measure of the extent to which clusters contain objects of a particular class. It's calculated as the average of the maximum number of elements of any class within each cluster.
- **Personas**: Representative archetypes of users or customers, created to help teams understand and cater to different user needs and behaviors.
- **Intelligent Agent**: A system that perceives its environment and takes actions that maximize its chances of success.
- **Logic Reasoning**: A cornerstone of Artificial Intelligence; the process of using rules of deduction to derive a conclusion from given assumptions.
- **Propositional Logic**: The simplest logic, illustrating basic ideas using propositional symbols and logical connectives to form sentences.
- **Satisfying Assignment**: An assignment of truth values to variables in a propositional formula that makes the formula evaluate to true.
- **Boolean Satisfiability Problem (SAT)**: The problem of determining whether a given propositional formula is satisfiable.
- **Literal**: A propositional variable or its negation.

- **Clause:** A disjunction of literals.
- **Conjunctive Normal Form (CNF):** A propositional formula that is a conjunction of clauses.
- **Valid Sentence:** A sentence that is true in all models (also known as a tautology).
- **Satisfiable Sentence:** A sentence that is true in at least one model.
- **Unsatisfiable Sentence:** A sentence that is true in no models.
- **3-Satisfiability (3-SAT):** A restricted version of the SAT problem where each clause contains exactly three literals.
- **Equisatisfiable:** Two formulas are equisatisfiable if they have the same set of satisfying assignments.
- **Classical Planning:** The process of finding a sequence of actions that achieves a desired goal.
- **Combinatorial Design:** The creation of complex mathematical structures with desirable properties, such as Latin squares or Steiner systems.
- **Recursion:** A process where a function calls itself, eventually terminating in a base case.
- **Base Case:** The simplest form of a recursive problem, providing a direct solution without further recursion.
- **Recursive Case:** The part of a recursive function or process that breaks down a larger problem into smaller, similar subproblems, each step closer to the base case.
- **Breadth First:** A graph traversal algorithm that visits all the nodes at the same level before moving to the next level.
- **Depth First:** A graph traversal algorithm that explores a branch as far as possible before backtracking.
- **Binary Search Tree:** A tree data structure where each node has at most two children, and the left child is less than the parent, while the right child is greater than the parent.
- **Search Tree:** A tree data structure used to represent the possible states and actions in a search problem.
- **State:** A description of the configuration of the environment in a search problem.
- **Action:** Activities that move from one state to another in a search problem.
- **Search Algorithm:** A procedure that determines which actions should be tried in which order to find a solution in a search problem.
- **Successor Function:** A function that maps a state to its possible successor states.
- **World State:** A complete description of every detail of the environment.
- **Search State:** A simplified representation of the environment, containing only the details needed for planning.
- **Rational Agent:** An agent that acts to maximize its expected goal achievement, given its available information.
- **SAT Solving:** The process of determining whether a Boolean formula is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula evaluate to true.
- **General Automated Reasoning:** A process that uses a model generator to create a model from a problem instance and then uses a general inference engine to find a solution.
- **Exponential Complexity Growth:** The rapid increase in computational complexity as the size of a problem increases, often characterized by an exponential function.

- **Partial Assignment:** An assignment of truth values to a subset of the variables in a Boolean formula, leaving some variables unassigned.
- **Simplification of a formula by an evaluated literal:** A process of reducing a CNF formula by assigning a truth value to a variable and removing clauses that become true or literals that become false.
- **Pure Literal Rule:** A rule in SAT solving that states that if a variable appears only positively or only negatively in a formula, it can be assigned a truth value that satisfies all clauses containing it.
- **Unit Propagation (BCP):** A technique in SAT solving that assigns truth values to variables based on unit clauses (clauses with only one literal).
- **Resolution Rule:** An inference rule in propositional logic that allows deriving a new clause from two clauses containing a literal and its negation.
- **Clause Learning:** A technique in SAT solving where the reason for a conflict is learned and added as a new clause to prevent similar conflicts in the future.
- **Heavy-tailed behavior:** A phenomenon in SAT solvers where the runtime distribution is skewed, with a few runs taking significantly longer than the average.
- **Hamiltonian Path Problem:** A problem of finding a path in a graph that visits each vertex exactly once.
- **Graph Coloring:** The problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color.
- **Disparate Treatment:** A type of discrimination where decisions are (partly) based on a subject's protected attribute information.
- **Disparate Impact:** A type of discrimination where outcomes disproportionately hurt (or benefit) people with certain sensitive attribute values.
- **Bias in Data Models:** The unintentional perpetuation of existing societal biases in algorithms, particularly those that learn from data.
- **Proxies for Discrimination:** Seemingly neutral data points that can act as stand-ins for discriminatory factors, such as race, gender, or religion.
- **Anonymized Data:** Data that has had identifying information removed; however, it may still be possible to re-identify individuals.
- **Privacy:** The state of being free from unwanted observation or intrusion; in data science, it refers to the protection of personal information.
- **Security:** Measures taken to protect data from unauthorized access, use, disclosure, disruption, modification, or destruction.
- **Fairness through Blindness:** A method to mitigate bias in algorithms by ignoring irrelevant or protected attributes during model training.
- **Group Fairness:** A fairness criterion that aims to equalize outcomes for different groups, such as minorities and majorities.
- **Individual Fairness:** A fairness criterion that aims to treat similar individuals similarly, regardless of group membership.
- **Interpretable Results:** Results from an AI algorithm that are easily understandable and explainable, increasing trust and transparency.

2 Problem Types

- **Understanding the Nature of Intelligence:** This problem explores the definition and characteristics of intelligence, both human and artificial. Different perspectives and definitions of AI are examined.
- **Evaluating Intelligent Behavior:** This problem focuses on developing methods for assessing whether a machine exhibits intelligent behavior. The Turing Test and its limitations are discussed, along with counterarguments like Searle's Chinese Room argument.
- **Overcoming the Limits of Search:** This problem addresses the computational challenges of exhaustive search in complex games like chess and Go. The exponential growth of search space is highlighted, leading to the need for machine learning techniques to supplement search.
- **Addressing AI Safety and Ethical Concerns:** This problem explores the potential risks and ethical implications of AI systems. Examples include bias in algorithms and the vulnerability of neural networks to adversarial attacks.
- **Bridging the Gap Between Different AI Domains:** This problem investigates the commonalities and differences between seemingly disparate AI tasks, such as game playing and speech synthesis. The shared algorithmic components and challenges are analyzed.
- **Understanding the Challenges of Real-World AI Applications:** This problem examines the difficulties of applying AI to complex real-world scenarios, such as autonomous driving. The reasons for choosing autonomous vehicles as an AI Grand Challenge are explored.
- **Representing Linear Transformations with Matrices:** Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic:** The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for manipulating data in AI applications.
- **Solving Systems of Linear Equations:** Systems of linear equations can be represented and solved using matrices and their inverses. This is crucial for various AI tasks involving optimization and data analysis.
- **Matrix Transposition:** Matrix transposition is a fundamental operation that swaps rows and columns, useful for data manipulation and algorithm design in AI.
- **Understanding Perceptrons:** The perceptron, a fundamental building block of neural networks, is introduced, illustrating the concept of weighted inputs and activation functions.
- **Parallelizing Matrix Multiplication:** The slide demonstrates how matrix multiplication can be parallelized using domain decomposition, improving computational efficiency in AI applications.
- **Calculus in AI:** The slide highlights the importance of calculus, specifically derivatives, partial derivatives, and gradient calculations, in AI algorithms, particularly in optimization problems.
- **Calculating the Derivative of the Sigmoid Function:** The slide shows the step-by-step derivation of the derivative of the sigmoid function using calculus rules, a crucial step in training neural networks.
- **Linear Regression:** Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression:** Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting:** Fitting a curve to a set of data points, potentially using polynomial functions of higher degrees.

- **Overfitting:** The phenomenon where a model fits the training data too closely, resulting in poor generalization to unseen data.
- **Binary Classification:** Classifying data points into one of two categories using a linear model and a suitable loss function.
- **Choosing a Loss Function:** Selecting an appropriate loss function (e.g., 0-1 loss, perceptron loss, hinge loss, logistic loss) for binary classification, considering differentiability and computational efficiency.
- **Gradient Descent:** An iterative optimization algorithm used to minimize the loss function by following the direction of the negative gradient.
- **Stochastic Gradient Descent:** A variant of gradient descent that updates the model parameters using a randomly selected subset of the data points at each iteration.
- **Image Classification:** The problem of assigning predefined labels (e.g., cat, dog, truck) to images.
- **Addressing the Semantic Gap:** The challenge of bridging the gap between a computer's numerical representation of an image and a human's semantic understanding of it.
- **Handling Viewpoint Variation:** The difficulty of classifying images of the same object from different viewpoints.
- **Addressing Illumination Challenges:** The problem of classifying images of the same object under varying lighting conditions.
- **Overcoming Deformation Issues:** The challenge of classifying images of the same object in different poses or shapes.
- **Managing Occlusion Problems:** The difficulty of classifying images where parts of the object are hidden or obscured.
- **Reducing Background Clutter:** The challenge of classifying images where the object of interest is surrounded by distracting elements.
- **Finding Invariant Features:** The need to identify features that remain consistent across different variations of an object (viewpoint, lighting, pose).
- **Object Classification with Bag of Words:** Using the frequency of "visual words" (features) to classify objects in images.
- **Document Retrieval:** Representing documents using the frequencies of words from a dictionary.
- **Noise Reduction:** Methods for reducing noise (e.g., salt-and-pepper, impulse, Gaussian) in images.
- **Image Filtering:** Computing a function of the local neighborhood at each pixel to enhance or extract information from an image.
- **Blurring:** Smoothing an image by averaging pixel values in a local neighborhood.
- **Sharpening:** Enhancing image details by reversing the effects of blurring.
- **Bag of Features Object Categorization:** This problem focuses on using Bag of Features to categorize objects by mapping their feature vectors into a model space. The process involves extracting features from images, representing them as codewords, and using their frequencies to create category models.
- **Edge Detection:** This problem involves identifying edges in images, which are curves where brightness changes significantly. Edges are useful for object recognition, understanding 3D structure, and grouping pixels into objects.

- **Corner Detection:** This problem focuses on identifying corners in images, which are points where the image intensity changes significantly in multiple directions. The main idea is that translating a window across the image should cause large differences in patch appearance at corners.
- **Vision Pyramids:** This problem involves representing an image as a "pyramid" of progressively smaller and blurred versions. This hierarchical representation allows for feature extraction at different scales and resolutions.
- **Object Detection and Recognition using CNNs:** Convolutional Neural Networks are used to identify and locate objects within images and videos. Examples include identifying faces, cars, fruits, and handwritten digits.
- **Improving ImageNet Classification Accuracy:** This problem focuses on improving the accuracy of image classification models over time, as demonstrated by the decreasing error rate in the ImageNet challenge.
- **Comparing Dense and Convolutional Neural Networks:** This problem involves comparing the architectures and processing methods of standard fully connected neural networks and convolutional neural networks.
- **Understanding Artificial Neurons:** This problem focuses on understanding the fundamental operations of an artificial neuron, including weighted input summation and activation functions.
- **Convolution and Pooling Operations in CNNs:** This problem involves understanding the mathematical operations of convolution and pooling layers in CNNs, including filter application and downsampling.
- **Explaining Sparse Connectivity in Convolutional Layers:** This problem focuses on explaining why convolutional layers use sparse connectivity instead of fully connected layers, highlighting the benefits of local receptive fields and parameter sharing.
- **Understanding the ReLU Activation Function:** This problem involves understanding the mathematical definition and graphical representation of the Rectified Linear Unit (ReLU) activation function.
- **Understanding Pooling Layers:** This problem focuses on understanding the downsampling operation performed by pooling layers in CNNs, including max pooling and average pooling.
- **Backpropagation for Neural Network Training:** This problem involves understanding the backpropagation algorithm used to train neural networks by adjusting weights to minimize error.
- **Understanding a Simple CNN Structure:** This problem involves understanding the architecture of a simple CNN, including the sequence of convolutional, activation, pooling, and fully connected layers.
- **Visualizing Learned Convolution Filters:** This problem involves visualizing and interpreting the learned convolution filters from a trained CNN.
- **Biological Inspiration for Artificial Neural Networks:** This problem involves understanding the biological inspiration behind artificial neural networks, drawing parallels between biological neurons and perceptrons.
- **Using the MNIST Dataset for Handwritten Digit Recognition:** This problem involves using the MNIST dataset to train and evaluate models for handwritten digit recognition.
- **Using the CIFAR-10 Dataset for Image Classification:** This problem involves using the CIFAR-10 dataset to train and evaluate models for image classification and comparing the performance of different models.
- **Hyperparameter Tuning:** Adjusting hyperparameters to optimize model performance. This involves using a held-out validation set to find the best hyperparameter values.

- **Underfitting:** A model that performs poorly on both training and test data, indicating it is too simple to capture the underlying patterns in the data.
- **The Data Analysis Pipeline:** The process of collecting data, designing a model, inferring parameters, and applying the model, highlighting potential issues at each stage.
- **S-fold Cross Validation:** A technique to evaluate model performance and choose hyperparameters by splitting the data into S parts, using each part as a validation set once.
- **Matrix Factorization:** Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction:** Transforming data from a higher number of dimensions to a lower number of dimensions to discard irrelevant features, visualize high-dimensional data, or reflect the intrinsic dimensionality.
- **Low Rank Matrix Approximations:** Reducing the rank of a matrix to achieve dimensionality reduction, using equivalent definitions based on linearly independent subsets of columns/rows or matrix factorization.
- **Singular Value Decomposition (SVD):** Decomposing a matrix into three simpler matrices (U, S, V) to reveal latent factor affinities for entities and attributes.
- **Principal Component Analysis (PCA):** A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data.
- **Recommender System Algorithms:** Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Collaborative Filtering:** Predicting a user's rating for an item based on the ratings of similar users.
- **Modeling Systematic Biases:** Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Bias-Variance Tradeoff:** Balancing model complexity (number of parameters) and prediction accuracy (RMSE) to optimize the performance of a recommender system.
- **Clustering Data:** Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. Clustering is a common form of unsupervised learning.
- **Supervised vs. Unsupervised Learning:** Supervised learning uses paired inputs/outputs to learn a function, while unsupervised learning uses only inputs to learn a function to simplify the data. Examples of unsupervised learning include clustering and matrix factorization.
- **Choosing the Number of Clusters (k) in K-means:** Determining the optimal number of clusters (k) in K-means clustering by minimizing the sum of squared distances from each point to its cluster centroid (inertia). A visual approach involves plotting inertia against k and selecting the "elbow" point.
- **Evaluating Clustering Results Subjectively:** Assessing the quality of clusters by visually inspecting the reordered proximity matrix (heatmap) for a clear block pattern, and by visualizing clusters using PCA to reduce dimensionality and coloring points by cluster membership.
- **Evaluating Clustering Results Objectively:** Evaluating clusters using known class labels, such as using purity to measure the extent to which clusters contain objects of a particular class, or by comparing an observed similarity matrix (based on cluster membership) to an ideal similarity matrix (based on class values) using correlation.
- **K-means Algorithm:** An iterative algorithm that partitions data points into k disjoint subsets by repeatedly assigning points to the nearest cluster mean (centroid) and then recomputing the centroids.

- **Measuring Distance Between Data Points:** Using distance measures like Euclidean distance for continuous data and Hamming distance or matching coefficient for discrete data to quantify the dissimilarity between data points.
- **Solving the Satisfiability Problem (SAT):** This problem focuses on determining whether a given Boolean formula can be made true by assigning appropriate truth values to its variables.
- **Converting Logical Expressions to Conjunctive Normal Form (CNF):** This involves transforming arbitrary logical expressions into a standardized format (CNF) consisting of conjunctions of clauses, where each clause is a disjunction of literals.
- **Determining Validity and Satisfiability of Logical Sentences:** This problem involves classifying logical sentences as valid (true in all models), satisfiable (true in at least one model), or unsatisfiable (false in all models).
- **Applying SAT to Model Checking:** This involves using SAT solvers to verify the correctness of hardware or software systems by encoding system properties and transitions into a Boolean formula and checking its satisfiability.
- **Applying SAT to Classical Planning:** This involves using SAT solvers to find plans that achieve a goal state by encoding the planning problem as a Boolean formula and finding a satisfying assignment.
- **Applying SAT to Combinatorial Design:** This involves using SAT solvers to find complex mathematical structures with desirable properties, such as Latin squares or Steiner systems.
- **Applying SAT to Diverse Domains:** This involves using SAT solvers to solve sub-problems in various domains, including test pattern generation, scheduling, optimal control, protocol design, and multi-agent systems.
- **Reducing SAT to 3-SAT:** This involves transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals. This is done to leverage the efficiency of algorithms designed for 3-SAT.
- **Representing Recursion:** This problem focuses on understanding and implementing recursive algorithms and data structures, such as the Fibonacci sequence and tree traversal.
- **Problem Solving as Search:** This problem involves formulating problems as search problems, defining states, actions, and goal tests, and choosing appropriate search algorithms. Examples include mazes and the 8-puzzle.
- **Optimizing Search Algorithms:** This problem focuses on improving the efficiency of search algorithms by avoiding redundant computations and managing the state space effectively. The Fibonacci sequence is used as an example to illustrate the concept of optimization.
- **Representing Problems as Graphs:** This problem involves representing problems as graphs, where nodes represent states and edges represent actions. Examples include pathfinding problems and the 8-puzzle.
- **Defining State Spaces:** This problem focuses on defining appropriate state spaces for different problems, distinguishing between world states and search states, and considering the size of the state space. The Pac-Man game is used as an example.
- **Designing Rational Agents:** This problem involves designing agents that act rationally to maximize goal achievement given available information. The concept of rational behavior is discussed in relation to agent design.
- **General Automated Reasoning:** Developing better reasoning and modeling technology to solve problems across various domains, such as logistics, chess, planning, and verification.

- **Overcoming Exponential Complexity Growth:** Addressing the challenge of exponentially increasing computational complexity as problem size grows in domains like VLSI verification, military logistics, and chess.
- **Evaluating Progress in SAT Solving:** Assessing the significant advancements in SAT solving capabilities over the past two decades, considering factors beyond Moore's Law.
- **Understanding Bounded Model Checking (BMC):** Exploring the problem of verifying temporal properties in models (e.g., hardware designs) by reducing it to a propositional satisfiability problem.
- **SAT Encoding:** Translating problem specifications into a standard Conjunctive Normal Form (CNF) format suitable for SAT solvers.
- **Understanding SAT Solvers as Search Engines:** Analyzing the efficiency of modern SAT solvers in searching for solutions, considering metrics like decisions, conflicts, and learned clauses per second.
- **Analyzing SAT Solver Progress:** Examining the historical improvement in SAT solver performance over time, using benchmark data to illustrate the advancements.
- **Defining Satisfiability of CNF Formulas:** Formally defining the conditions for satisfiability of a CNF formula and illustrating how partial assignments can simplify the verification process.
- **Utilizing Partial Assignments in SAT Solving:** Employing partial assignments to variables to incrementally construct solutions and simplify the SAT problem.
- **Simplifying CNF Formulas:** Describing techniques for simplifying CNF formulas based on the truth value of a specific variable to reduce the search space.
- **Understanding Principles of SAT Solving Algorithms:** Outlining the core principles of recursive SAT solving algorithms, including argument modification and pre-checks for efficiency.
- **Implementing a Naïve SAT Solver:** Presenting a basic recursive SAT solving algorithm that uses backtracking and simplification.
- **Applying the Pure Literal Rule:** Describing a simplification rule for SAT problems where a variable appears only positively or negatively.
- **Implementing Unit Propagation:** Explaining the unit propagation technique (BCP) for efficiently solving SAT problems by identifying and assigning values to literals in unit clauses.
- **Implementing the DPLL Algorithm:** Presenting the DPLL algorithm, a recursive SAT solving algorithm that incorporates BCP and backtracking.
- **Visualizing DPLL Search:** Illustrating the search space explored by the DPLL algorithm using implication graphs.
- **Applying the Resolution Rule:** Describing the resolution rule for deriving new clauses from existing ones in a CNF formula.
- **Applying Clause Learning:** Explaining the technique of clause learning to improve the efficiency of SAT solvers by learning from conflicts during the search.
- **Understanding Key Advances in SAT Reasoning:** Summarizing key advancements in SAT solving, including the use of machine learning for solver selection and automatic parameter tuning.
- **Applying SAT to Real-World Problems:** Introducing the application of SAT solvers to real-world problems in various domains.
- **Applying Cook's Theorem to SAT:** Explaining Cook's Theorem and its implications for the broad applicability of SAT solvers to problems in NP.

- **Encoding the Hamiltonian Path Problem into SAT:** Describing the process of encoding the Hamiltonian Path Problem, a graph problem, into a SAT formula.
- **Encoding Graph Coloring into SAT:** Describing the process of encoding the graph coloring problem into a SAT formula.
- **Bias and Fairness in Data:** This problem explores the presence of bias in data and its impact on the fairness of AI systems, including the perpetuation of existing societal biases.
- **Why data analysis methods are helpful:** This problem examines the advantages and disadvantages of using data analysis methods over human methods, considering factors like cost, scalability, accuracy, and fairness.
- **Data Privacy and Anonymization:** This problem investigates the challenges of protecting individual privacy when using data for AI applications, highlighting the limitations of anonymization techniques and the potential for re-identification.
- **Algorithmic Bias and Discrimination:** This problem focuses on the ways in which algorithms can inadvertently reflect and amplify existing societal biases, leading to discriminatory outcomes in various applications such as lending, advertising, and criminal justice.
- **Interpretability and Trustworthiness of AI Models:** This problem addresses the challenges of understanding how AI models arrive at their predictions and the importance of building trust in AI systems through interpretability and transparency.
- **Addressing Bias in AI Models:** This problem explores methods for detecting and mitigating bias in AI models, including techniques for constraining algorithms, achieving group or individual fairness, and understanding when model outputs are unreliable.
- **The Legal and Ethical Implications of AI:** This problem examines the legal and ethical considerations surrounding the use of AI, including anti-discrimination laws, privacy concerns, and the need for responsible AI development and deployment.
- **Limitations of AI in High-Stakes Applications:** This problem discusses the challenges of using AI in high-stakes domains like healthcare, where the potential consequences of errors can be severe, highlighting the trade-offs between accuracy and risk.

3 Algorithm Solutions

- **Minimax Search:** A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning:** An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Matrix Addition:** $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication:** $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication:** $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Transposition:** Swapping rows and columns of a matrix A to obtain A^T .
- **Solving Linear Equations with Invertible Matrices:** $Ax = b \Rightarrow x = A^{-1}b$
- **Derivative of Sigmoid Function:** $\frac{d}{dx} \sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$

- **Multiple Linear Regression:** Find the optimal coefficients β that minimize the sum of squared errors between the observed and predicted values. This is done by solving the equation $\beta = (X^T X)^{-1} X^T y$, where X is the design matrix, y is the vector of observed values, and β is the vector of coefficients.
- **Perceptron Algorithm:** Iteratively update the weight vector w by adding $y_n x_n$ if the data point x_n is misclassified, where y_n is the true label and x_n is the feature vector. If the data is linearly separable, this algorithm will find a separating hyperplane.
- **Logistic Regression:** Minimize the logistic loss function $F(w) = \sum_{i=1}^N \log(1 + e^{-y_i w^T x_i})$ using stochastic gradient descent. The update rule is $w \leftarrow w + \alpha(\sigma(-y_n w^T x_n) y_n x_n)$, where α is the learning rate and σ is the sigmoid function.
- **Bag of Features:** Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier:** Assign label of nearest training data point to each test data point
- **Moving Average Filter:** Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average Filter:** Add weights to the moving average to give different importance to neighboring pixels
- **Image Sharpening:** $f_{sharp} = f + \alpha(f - f_{blur})$
- **Finite Difference:** $\frac{df}{dx}[x, y] \approx F[x + 1, y] - F[x, y]$
- **Image Gradient Magnitude:** $\|\nabla f\| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction:** $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Sum of Squared Differences (SSD):** $E(u, v) = \sum_{(x,y) \in W} [I(x + u, y + v) - I(x, y)]^2$
- **Anisotropic Gaussian:** $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2}) + (\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian:** $G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{x^2 + y^2}{2\sigma^2}}$
- **Convolution:** A convolution operation involves sliding a filter (kernel) across an input image, performing element-wise multiplication between the filter and the corresponding input region, and summing the results to produce a single output value. This process is repeated for every possible position of the filter on the input image, resulting in an output feature map.
- **Max Pooling:** This operation involves dividing the input feature map into non-overlapping regions (e.g., 2x2 squares) and selecting the maximum value within each region as the output. This reduces the spatial dimensions of the feature map while retaining the most prominent features.
- **Average Pooling:** This operation involves dividing the input feature map into non-overlapping regions (e.g., 2x2 squares) and calculating the average value within each region as the output. This reduces the spatial dimensions of the feature map while retaining a summary of the features.
- **ReLU Activation Function:** This function outputs the input if it is positive, and 0 otherwise. Mathematically, it is defined as $f(x) = \max(0, x)$
- **Backpropagation:** This algorithm calculates the gradient of the loss function with respect to the network's weights and uses this gradient to update the weights iteratively, aiming to minimize the loss and improve the network's accuracy.
- **Regularized Linear Regression:** $E(w) = RSS(w) + \lambda R(w)$, where $R(w) = \|w\|_2^2 = w_1^2 + \dots + w_k^2$ or $R(w) = \|w\|_1 = |w_1| + \dots + |w_k|$.

- **S-fold Cross Validation:** Split training data into S equal parts; use each part as validation data, others as training data; choose hyperparameter with best average performance.
- **Singular Value Decomposition (SVD):** $A = USV^T$, where A is an $n \times d$ matrix, U is an $n \times n$ orthogonal matrix, V is a $d \times d$ orthogonal matrix, and S is an $n \times d$ diagonal matrix with nonnegative entries sorted from high to low.
- **Principal Component Analysis (PCA):** A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data. The eigenvectors of the covariance matrix correspond to the principal components.
- **Modeling Systematic Biases in Recommender Systems:** $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where μ is the overall mean rating, b_i is the bias for user i , b_j is the bias for movie j , and $u_i \cdot v_j$ represents the user-movie interaction.
- **K-means Clustering:** The algorithm iteratively assigns data points to the nearest cluster centroid, then recalculates the centroids as the mean of the points in each cluster, repeating until convergence.
- **Euclidean Distance:** $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$, where x and y are p -dimensional vectors.
- **Hamming Distance:** Counts the number of differing bits between two binary vectors.
- **Matching Coefficient:** Hamming distance normalized by the number of attributes/bits.
- **Purity:** $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^K \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of points of class j in cluster i .
- **Conversion to CNF:** This algorithm transforms a logical expression into Conjunctive Normal Form (CNF) by applying logical equivalences to eliminate implications, biconditionals, and negations, and then using distributivity to flatten the expression into a conjunction of clauses.
- **Satisfiability Problem (SAT):** This problem determines if a given propositional formula (often in CNF) is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **3-Satisfiability (3-SAT):** This problem is a restricted version of SAT where each clause in the formula contains exactly three literals. It's shown that any SAT problem can be reduced to a 3-SAT problem by introducing new variables and creating equivalent clauses with three literals.
- **Recursive Problem Solving:** Do part of the work and reduce the rest into a similar but smaller problem.
- **Breadth First Search:** Visits nodes level by level. Check the current node, add children to a queue.
- **Depth First Search:** Traverses down a branch. Check current node, then left, then right (other orders are possible).
- **Dynamic Programming (Fibonacci Optimization):** Store and reuse previously calculated Fibonacci numbers to avoid redundant calculations.
- **Naïve Solver:** A recursive algorithm that explores all possible variable assignments to determine satisfiability of a propositional formula. It checks for immediate satisfiability or unsatisfiability, then recursively calls itself with assignments of true and false to an unassigned variable, backtracking if both fail.
- **DPLL Algorithm:** A recursive algorithm that extends the Naïve Solver by incorporating Boolean Constraint Propagation (BCP) to efficiently identify and resolve unit clauses, significantly reducing the search space.
- **BCP (Boolean Constraint Propagation):** A sub-algorithm that repeatedly identifies and assigns values to literals in unit clauses (clauses with only one unassigned literal), creating a cascade of assignments until either a conflict is found or no more unit clauses exist.

- **Resolution Rule:** An inference rule that derives a new clause $()$ from two clauses (x) and $(\neg x)$ containing a literal (x) and its negation $(\neg x)$.
- **Clause Learning:** A technique used in advanced SAT solvers where, upon backtracking from a conflict, a new clause is learned to prevent similar conflicts in the future. This clause represents the "reason" for the conflict, effectively pruning the search space.
- **Fairness through Blindness:** Ignore protected attributes during model training and prediction.
- **Group Fairness:** $P(outcome|S) = P(outcome|T)$, where S and T are two groups. This is insufficient as a notion of fairness.
- **Individual Fairness:** Treat similar individuals similarly; similar individuals should be treated similarly for the purpose of the classification task and should be similarly distributed over outcomes.
- **Detecting and removing bias from models:** Keep bias features in data during training, remove what was learned from these bias features after training.